PERFORMANCE EVALUATION OF SEQUENTIAL AND PARALLEL MATRIX MULTIPLICATION

Individual Assignment 03 Experimental Report

Rubén Perdomo Aspas

Repository: https://github.com/RubenPerdomo159/BigData-Individual-Assigments.git

1. Introduction and Objectives.

Matrix multiplication is a demanding operation used in scientific computing, data analysis, and machine learning. For two dense square matrices A and B, each with dimensions n × n, the standard algorithm requires $O(n^3)$ arithmetic work. Each output row can be computed separately, which allows for shared-memory parallel execution.

The main goal of this assignment is to explore how the number of threads affects execution time, speedup, and parallel efficiency. Another goal is to see how problem size impacts the value of parallel execution and to look at the extra costs introduced by the parallel implementation, including its single-thread setup.

2. Implementation and Methodology.

2.1. Algorithm.

Both implementations use the traditional triple-loop algorithm. The loops go through output row i, output column j and shared dimension k. Each output element is computed as $C[i,j] = \Sigma(k=0 \text{ to } n-1)\ A[i\ k] \times B[k,j]$. The input and output arrays have float64 values.

The version that runs in sequence carries out the row loop normally. The version that runs in parallel uses Numbas prange to split the iterations of the row loop across threads. Each thread writes to a row of C so the version that runs in parallel makes sure there are no simultaneous writes, to the same output elements. The version that runs in parallel has the number of Numba threads set manually for each benchmark.

2.2. Input Data and Correctness.

The benchmark uses deterministic matrix generation. This means the same input values are used for both the parallel methods at each matrix size. Before running the benchmark correctness tests are performed. The raw measurements include a correctness field and a checksum for every execution. All 125 measurements show =true. The checksums match between the parallel methods, for each matrix size.

2.3. Experimental Protocol.

In this experiment we use matrix dimensions n = 100, 200 300 400 and 500. The sequential baseline is measured per repetition ensuring that each repetition captures a single baseline measurement. The parallel implementation is measured with 1, 2 4 and 8 threads and the experiment records the performance, for each thread count. One warm-up run is performed before measurement. Then five repetitions are recorded for each configuration to provide stable data. Compilation and warm-up are excluded from the measured kernel time so only the core matrix multiplication kernel is timed. Timing

uses Pythons monotonic high-resolution perf_counter_ns timer and the measured region covers the matrix multiplication kernel.

3. Experimental Environment.

Operating System	Windows 11
Architecture	AMD64
Processor	Intel64 Family 6 Model 140 Stepping 1 GenuineIntel
Logical Processors	8
Python	3.13.12 (64-bits)
NumPy	2.4.4
Numba	0.68.1
Maximum Numba threads	8

The environment file does not identify the commercial CPU model, the physical core count, the installed RAM or the CPU affinity. These details should be added before the environment file is submitted if they can be obtained from the machine. Therefore the current data confirms eight processors but is insufficient to determine whether eight benchmark threads exceed the number of physical cores.

4. Performance Metrics.

For each setup the middle value of the five recorded run times is taken as the running time. The spread of the data is shown by the interquartile range which is calculated as IQR = Q3 - Q1. The speedup is found by dividing the middle run time by the middle run time when using p threads (S(p) = Tseq/Tp). The parallel efficiency is E(p) = S(p)/p, which is the speedup divided by the number of threads. When the efficiency is than 1 it shows a speedup that is more than linear. This should be looked at carefully because it might be due to changes in how the time's measured or differences, in how the program runs instead of breaking any rules.

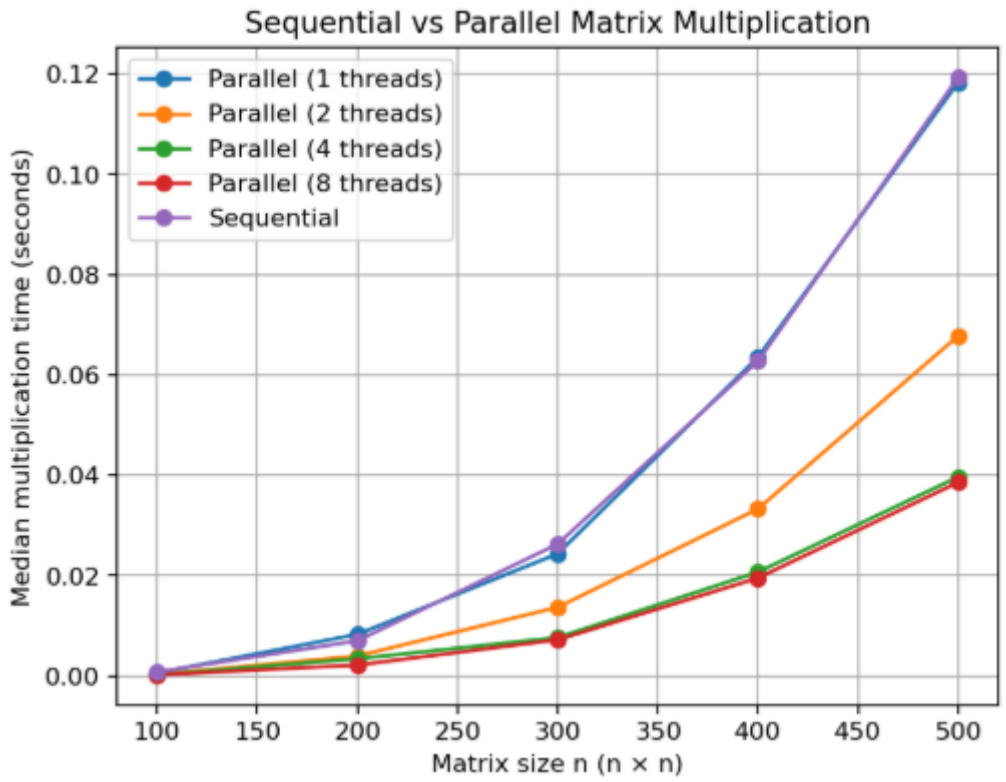
5. Results.

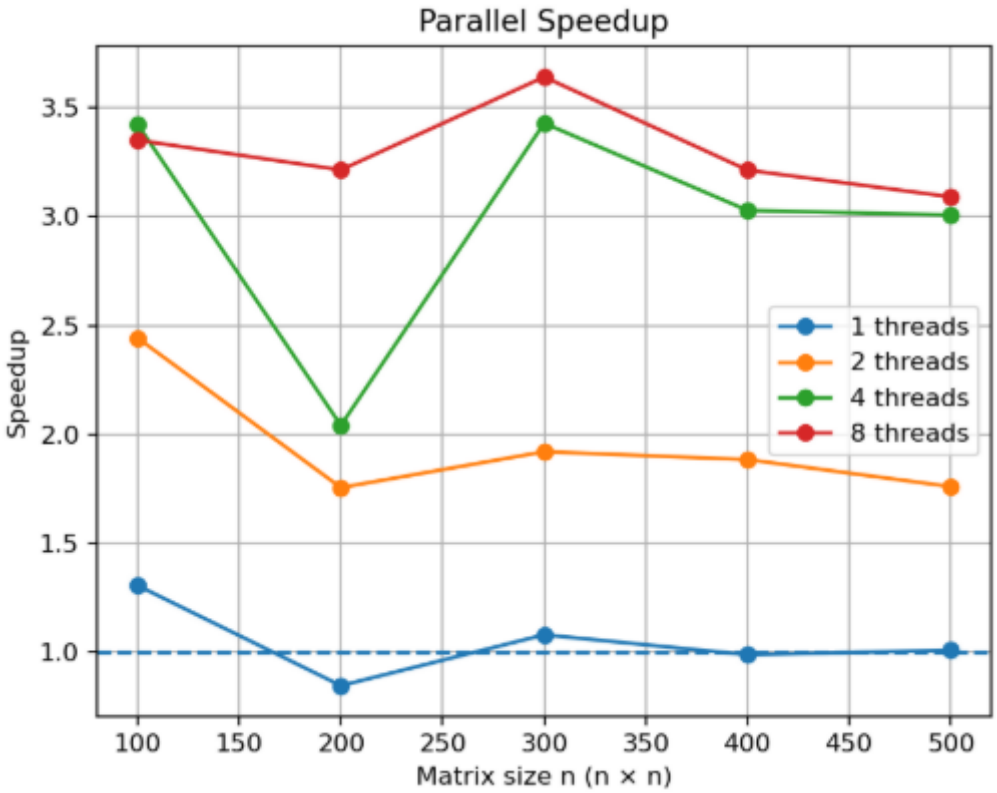
This table reports median execution times in milliseconds. The sequential column is the baseline. Parallel columns show the median time for each thread count.

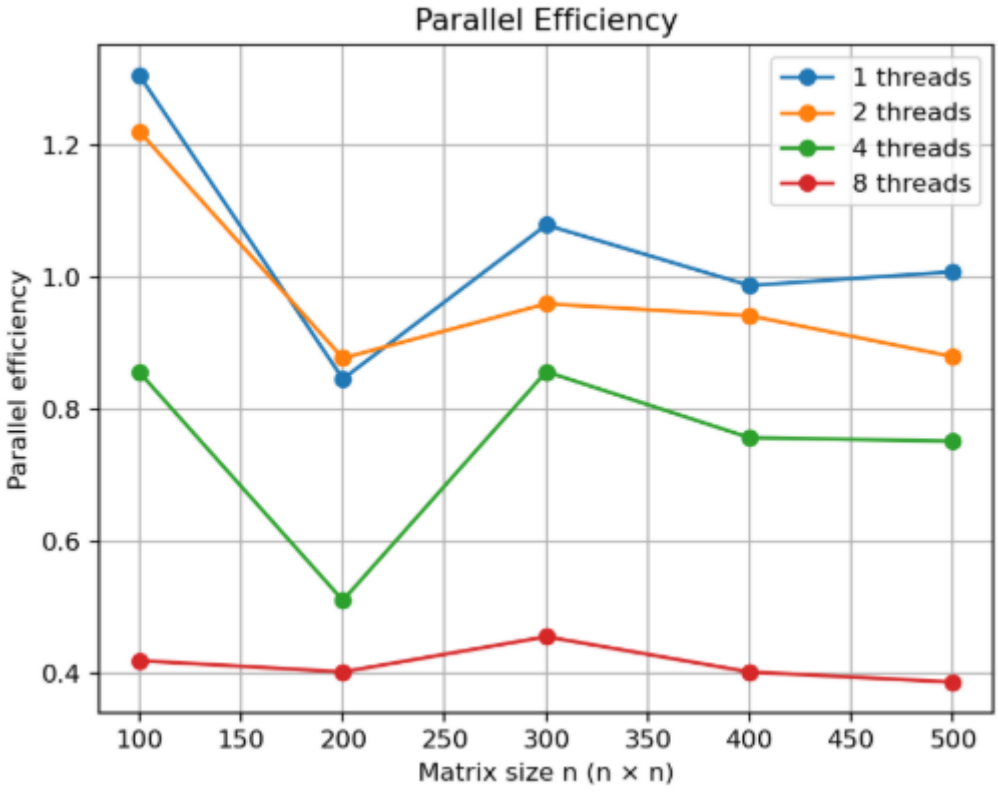
Matrix n	Sequential	Parallel 1T	Parallel 2T	Parallel 4T	Parallel 8T
100	0.815	0.624	0.334	0.238	0.243
200	7.036	8.316	4.008	3.448	2.189
300	26.330	24.388	13.712	7.680	7.232
400	62.781	63.553	33.322	20.734	19.540
500	119.343	118.319	67.817	39.690	38.622

The next table presents the speedup and efficiency calculated for each parallel configuration. The sequential baseline has no speedup or efficiency value.

Matrix n	Threads	Speedup	Efficiency
100	1	1.306	1.306
100	2	2.442	1.221
100	4	3.424	0.856
100	8	3.350	0.419
200	1	0.846	0.846
200	2	1.755	0.878
200	4	2.041	0.510
200	8	3.214	0.402
300	1	1.080	1.080
300	2	1.920	0.960
300	4	3.428	0.857
300	8	3.641	0.455
400	1	0.988	0.988
400	2	1.884	0.942
400	4	3.028	0.757
400	8	3.213	0.402
500	1	1.009	1.009
500	2	1.760	0.880
500	4	3.007	0.752
500	8	3.090	3.086




Parallel Speedup
Speedup
Matrix size n (n × n)
1 threads
2 threads
4 threads
8 threads
3.5
3.0
2.5
2.0
1.5
1.0
100
150
200
250
300
350
400
450
500


Parallel Efficiency
Parallel efficiency
Matrix size n (n × n)
1 threads
2 threads
4 threads
8 threads
1.2
1.0
0.8
0.6
0.4
100
150
200
250
300
350
400
450
500

6. Discussion.

The results show that the benefit of execution depends on matrix size. For n = 100 the measured configuration is four threads, with a speedup of approximately 3.42. Increasing the count to eight threads does not improve the result further. For n = 200 eight threads provide the measured speedup (about 3.21) although the four-thread measurements show unusually high variability. For n = 300 and n = 400 eight threads also give the median runtime with speedups of approximately 3.64 and 3.21 respectively. For n = 500 eight threads give a speedup of 3.09 only slightly better than four threads (approximately 3.01).

The one-thread parallel configuration is useful for observing the overhead associated with using the implementation even when only one thread is active. Its performance is close to the baseline for n = 300, n = 400 and n = 500 while it is slower, for n = 200 and faster for n = 100. These differences show that runtime overhead and measurement variation can be significant for smaller workloads.

The speedup is sublinear for multi-thread configurations: doubling the number of threads does not halve execution time. This is expected because parallel execution has scheduling and coordination costs and the available memory subsystem and processor resources impose limits. At n = 500 moving from four to eight threads reduces the median from about 39.69 ms to 38.62 ms, a small improvement. This suggests diminishing returns at the tested thread count for this workload and machine.

The n = 200 measurements include outliers, for four and eight threads. The median and IQR reduce the influence of values but the variation should still be acknowledged. A quiet machine, repeated runs and consistent CPU conditions are important, for benchmarking.

7. Memory Use and Limitations.

When I look at the CSV the memory column tells me the estimated payload of the three dense float64 matrices A, B and C. That payload is calculated as $3 * n^2 * 8$ bytes. The memory column is not a measurement of the peak process memory or the total amount of RAM that the program uses. It also leaves out Python, NumPy and Numba runtime overhead thread stacks and other allocations. Because of this the report treats the memory column value as the size of the matrix payload not as the memory usage.

The experiment is limited to one machine, one data type, one matrix multiplication algorithm and five matrix sizes. Five repetitions are carried out for each configuration. The exact CPU model, core count, installed RAM and CPU affinity are missing from the captured environment file. These omissions limit the ability to interpret hardware details especially when we talk about oversubscription.

8. Conclusion.

This experiment shows that using shared-memory parallelism can significantly cut down the time it takes to multiply matrices especially when the matrices get bigger. The fastest setup changes depending on how large the problem's. At first adding more threads helps a lot but the gains start to slow down after a point. For the matrices the eight-thread setup usually works the best.. Even then it doesn’t give an eightfold speedup. That means you can’t just assume more threads always make things faster, in proportion. Instead the right number of threads should be picked based on both the size of the workload and the hardware you have.